Namespace _208_Group_Project_Demo

Classes

Blackjack

Represents a game of blackjack, managing the card shoe, player and dealer hands, and game logic.

Card

CardCounter

Calculates counts according to a HiLo card counting system

FrontEnd

Enums

BlackjackAction

BlackjackResult

BlackjackState

Rank

Suit

Class Blackjack

Namespace: _208_Group_Project_Demo

Assembly: 208_Group_Project_Demo.dll

Represents a game of blackjack, managing the card shoe, player and dealer hands, and game logic.

```
public class Blackjack
```

Inheritance

object ← Blackjack

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

Blackjack()

```
public Blackjack()
```

Properties

currentState

```
public BlackjackState currentState { get; }
```

Property Value

BlackjackState

playerHand

```
public List<Card> playerHand { get; }
```

Property Value

List <Card>

playerValue

```
public int playerValue { get; }
```

Property Value

int

Methods

DealCards(List<Card>, int)

Deals n cards to either the player or the dealer, specified with toPlayer

```
public List<Card> DealCards(List<Card> toHand, int n = 1)
```

Parameters

`toHand` List <Card>

The hand the cards are dealt to

`n` int

The number of cards to deal

Returns

List <Card>

The hand with cards dealt to it

DealerTurn()

Executes the dealer's turn in the Blackjack game, updating the dealer's hand and game state according to standard rules.

```
public void DealerTurn()
```

EndGame()

Ends the current blackjack game, determines the outcome based on player and dealer hands, and outputs the result.

```
public BlackjackResult EndGame()
```

Returns

BlackjackResult

A BlackjackResult value indicating the outcome of the game.

HandToString(List<Card>)

Returns a comma-separated string of card IDs from the given hand.

```
public static string HandToString(List<Card> hand)
```

Parameters

`hand` List <Card>

The list of Card objects representing the hand.

Returns

string

A string containing the IDs of the cards in the hand, separated by commas.

HandValue(List<Card>)

Calculates the total value of a hand of cards, treating aces as 1 or 11 to maximize the hand's value without exceeding 21.

```
public static int HandValue(List<Card> hand)
```

Parameters

`hand` List <Card>

The list of cards whose value is to be calculated.

Returns

int

The total value of the hand.

LookAtDealerHand()

```
public List<Card> LookAtDealerHand()
```

Returns

List <Card>

PlayBlackjack()

Plays a complete game of blackjack between the player and the dealer, handling all game logic and user interaction.

```
public BlackjackResult PlayBlackjack()
```

Returns

BlackjackResult

A BlackjackResult indicating the outcome of the game.

PlayerTurn(BlackjackAction)

Processes the player's action during their turn in a game of Blackjack.

```
public void PlayerTurn(BlackjackAction playerAction)
```

Parameters

`playerAction` BlackjackAction

The action chosen by the player, such as Hit or Stand.

ShoeCount()

```
public int ShoeCount()
```

Returns

int

StartBlackjack()

Initializes and starts a new game of blackjack, dealing cards to the player and dealer and updating the game state.

```
public void StartBlackjack()
```

Enum BlackjackAction

Namespace: [208_Group_Project_Demo]

Assembly: 208_Group_Project_Demo.dll

```
public enum BlackjackAction
```

Fields

`Hit = 0`

`Stand = 1`

Enum BlackjackResult

Namespace: 208_Group_Project_Demo

Assembly: 208_Group_Project_Demo.dll

```
public enum BlackjackResult
```

Fields

BothNatural = 2

DealerBust = 4

DealerNatural = 1

DealerWins = 5

PlayerBust = 3

PlayerNatural = 0

PlayerWins = 6

StandOff = 7

Enum BlackjackState

Namespace: _208_Group_Project_Demo

Assembly: 208_Group_Project_Demo.dll

```
public enum BlackjackState
```

Fields

`DealerTurn = 2`

`GameEnd = 3`

`NoGame = 0`

`PlayerTurn = 1`

Class Card

Namespace: _208_Group_Project_Demo

Assembly: 208_Group_Project_Demo.dll

```
public class Card
```

Inheritance

object ← Card

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

Card(Suit, Rank)

```
public Card(Suit _suit, Rank _rank)
```

Parameters

`_suit` Suit

`_rank` Rank

Properties

id

```
public string id { get; }
```

Property Value

string

rank

```
public Rank rank { get; }
```

Property Value

Rank

suit

```
public Suit suit { get; }
```

Property Value

Suit

value

```
public int value { get; }
```

Property Value

int

Methods

GetDeck()

```
public static Card[] GetDeck()
```

Returns

Card[]

ShuffleDeck(Card[])

```
public static Card[] ShuffleDeck(Card[] deck)
```

Parameters

`deck` Card[]

Returns

Card[]

Class CardCounter

Namespace: _208_Group_Project_Demo

Assembly: 208_Group_Project_Demo.dll

Calculates counts according to a HiLo card counting system

```
public class CardCounter
```

Inheritance

object ← CardCounter

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Constructors

CardCounter(Blackjack)

```
public CardCounter(Blackjack _game)
```

Parameters

`_game` Blackjack

Fields

game

```
public Blackjack game
```

Field Value

Blackjack

lastShoeCount

```
public int lastShoeCount
```

Field Value

int

Methods

GetRunningCount()

Calculates and returns the current running count based on the cards dealt to the player and dealer, resetting the count if the card shoe has been refilled.

```
public int GetRunningCount()
```

Returns

int

The current running count after processing new cards.

GetTrueCount()

Calculates and returns the true count by dividing the running count by the number of decks remaining in the shoe.

```
public double GetTrueCount()
```

Returns

double

The current true count as an integer.

Class FrontEnd

Namespace: 208_Group_Project_Demo

Assembly: 208_Group_Project_Demo.dll

```
public static class FrontEnd
```

Inheritance

object ← FrontEnd

Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

Methods

Input()

```
public static string Input()
```

Returns

string

Output(string)

```
public static void Output(string message)
```

Parameters

`message` string

Enum Rank

Namespace: 208_Group_Project_Demo

Assembly: 208_Group_Project_Demo.dll

```
public enum Rank
```

Fields

```
Ace = 0

Eight = 7

Five = 4

Four = 3

Jack = 10

King = 11

Nine = 8

Queen = 12

Seven = 6

Six = 5

Ten = 9

Three = 2

Two = 1
```

Enum Suit

Namespace: 208_Group_Project_Demo

Assembly: 208_Group_Project_Demo.dll

```
public enum Suit
```

Fields

`Club = 0`

`Diamond = 1`

`Heart = 2`

`Spade = 3`